REPORT · 2026年10月17日

兵马俑实机 S1 失败诊断与修复

COLMAP 0xC0000409 · 报表与 GUI 可读性

文保数字化研发组
2026年10月17日

目 录

一、事件概述与结论 4

1.1 本次交付的三类成果 4

二、失败证据链 5

2.1 failures.csv：崩溃点与退出码 5

2.2 metrics_summary.csv：只有一行 fail，耗时缺失 5

2.3 report.html：状态表空白 5

2.4 batch_gui.log：第二次运行成功注册 30/30 5

三、根因定位：分层排除 6

3.1 第 1 层：照片本身 6

3.2 第 2、3 层：中文路径与强制参数 6

3.3 第 4 层：数据库状态 6

3.4 关键对照实验：残留库 vs 空库 7

四、根因机理 8

4.1 代码缺陷 8

4.2 为什么 clean rebuild 也可能「看起来」成功 8

五、修复方案 9

5.1 全量重建前清库（核心） 9

5.2 COLMAP 输出完整落盘 9

5.3 失败信息可诊断 9

5.4 失败阶段记录真实耗时 9

5.5 修复清单 9

六、验证证据 11

6.1 干净库 S1 全流程实测 11

6.2 日志落盘实测 11

6.3 回归测试 11

6.4 失败报表实测渲染 11
七、报表与 GUI 可读性改造 12
7.1 报表前后对比 12
7.2 GUI 前后对比 12
八、本次实机结果的判读与后续操作建议 13
8.1 建议操作步骤 13
8.2 采集侧建议（30 张偏少） 13
九、工程约束沉淀 14

一、事件概述与结论

本次实机验证使用 **E:\兵马俑照片** 中 30 张尼康单反照片（原图 4496×3000，预处理后 2399×1601），走**场景②纯照片三维重建**。首次运行在 S1（COLMAP 位姿解算）第一步失败，报表中只有一行失败记录，**状态表完全空白、没有任何可视化产物。**

经分层排查，根因是 **S1 复用了上一次运行残留的 database.db**：残留库处于半写入状态（cameras=3、images 分属 camera 1/3），再次执行 feature_extractor 时触发 SQLite 约束冲突，COLMAP 内部断言失败后 abort，进程退出码 **3221226505（0xC0000409）**。同一批照片在**空库**上重跑 30/30 全部成功。

> 结论：这不是照片格式、中文路径或参数问题，而是**状态管理缺陷**——全量重建建立在了上一次运行的残留状态之上。修复后每次全量重建都会清空旧库，并从零产出。**附带修复**报表与 GUI 的可读性缺陷：S1 不在状态表中、章节编号重复、失败信息被压成一行、预览区无产物时不显示任何提示。

1.1 本次交付的三类成果

类别	内容	验证方式
根因修复	全量重建前清空 database.db / -wal / -shm / -journal 与 sparse 旧模型	空库实测 30/30，camera=1
可诊断性	每步 COLMAP 输出落盘 logs/colmap_stepN.log；失败信息含日志路径与诊断建议	日志文件实测生成，失败信息断言
可读性	报表纳入 S1、章节重编号、失败信息保留换行、新增诊断建议；GUI 预览回退中间产物	HTML 实测渲染 + 19 项新增测试

二、失败证据链

2.1 failures.csv：崩溃点与退出码

用户提供的 failures.csv 关键内容（原文节选）：

```
文物编号,阶段,异常类型,异常信息,时间
relic_001BMYyyyy,S1,SfmError,"COLMAP 步骤 1 失败 (exit=3221226505): 3]  Name: DSC_0011.JPG
I... feature_extraction.cc:282]  Dimensions:     2399 x 1601
I... feature_extraction.cc:285]  Camera:         #2 - SIMPLE_RADIAL
I... feature_extraction.cc:289]  Focal Length:   2878.80px
I... feature_extraction.cc:294]  Features:       14194 (SIFT)
Could not create logging file: No such file or directory
COULD NOT CREATE A LOGGINGFILE 20261017-105945.16360!
E... database_sqlite.cc:1220] SQLite error: constraint failed
",2026-10-17T10:59:51
```

三个关键信号：
① **exit=3221226505** 即 0xC0000409（STATUS_STACK_BUFFER_OVERRUN），在 COLMAP 中通常表现为内部 CHECK 失败后 abort，而不是普通的参数错误；
② **Camera: #2**——代码里写死了 **--ImageReader.single_camera 1**，正常情况下所有照片共用 Camera #1，出现 #2 说明数据库里已有相机记录；
③ **SQLite error: constraint failed**——数据库唯一约束冲突，直接指向重复插入。

2.2 metrics_summary.csv：只有一行 fail，耗时缺失

```
文物编号,阶段,状态,耗时(s),产物路径,关键指标
relic_001BMYyyyy,S1,fail,,,
```

失败阶段的耗时被记为 0（实际已运行约 96 秒）。这掩盖了「卡了一段时间才崩」这一事实，属于可诊断性缺陷。

2.3 report.html：状态表空白

报告第五章（失败清单）有内容，但第一章「各尊处理状态」表格**一行数据都没有**，第二章至第四章全部显示「未执行」。用户因此无法判断管线究竟跑到了哪一步。

2.4 batch_gui.log：第二次运行成功注册 30/30

用户重试后产生的日志显示：feature_extractor 30 张、耗时 81 秒并**正常完成**，随后 exhaustive_matcher、mapper 也完成。这直接排除了「照片本身无法被 COLMAP 处理」的可能，把怀疑范围收窄到**两次运行之间的状态差异**。

三、根因定位：分层排除

按「照片 → 路径 → 参数 → 状态」四层逐级排除，每一步都以实测为准，不做推测性结论。



图 1 S1 失败的分层排除路径（前三层各自以空库实测通过而排除）

3.1 第 1 层：照片本身

30 张照片全部为 2399×1601，尺寸完全一致；EXIF 在下采样时已剥离，不存在 Orientation 差异。**尺寸一致意味着 single_camera 不会因尺寸变化而新建相机**，进一步说明 Camera #2 只能来自数据库残留。

3.2 第 2、3 层：中文路径与强制参数

以空库、完全相同的命令重跑 feature_extractor：

```
colmap feature_extractor  --database_path E:/.../repro_clean/database.db  --image_path  E:/.../photos_small  --ImageReader.single_c

I... Processed file [30/30]  Name: DSC_0037.JPG
I...  Camera: #1 - SIMPLE_RADIAL
I... Elapsed time: 1.636 [minutes]
EXIT=0
```

结果：30/30、全部 Camera #1、退出码 0。**中文路径与参数均被排除。**

3.3 第 4 层：数据库状态

检查残留数据库（当时正在被第二次运行使用）：

```
cameras 3  images 30  keypoints 30  matches 389

camera 1 → DSC_0008, DSC_0009, DSC_0010, DSC_0011
camera 2 →  （无图像引用）
camera 3 → DSC_0012 起
```

图像按 camera 1 / camera 3 断裂分组，且 camera 2 成为孤儿记录——这是典型的**多次会话在同一库上追加写入**的结果。**Camera: #2** 正是崩溃日志里出现的那一条。

3.4 关键对照实验：残留库 vs 空库

实验	库状态	结果
A. 空库 + 30 张	不存在	30/30，全部 Camera #1，EXIT=0（1.6 分钟）
B. 残留库 + 30 张	cameras=3 / images=30	已有图像被跳过，EXIT=0（0.0 分钟）
C. 首次运行（实录）	半写入（部分图像）	第 4 张时 constraint failed → abort

实验 B 说明：**当残留库已经完整时**，COLMAP 会跳过已有图像而不崩；实验 C 说明：**当残留库处于半写入状态**（部分相机/图像已提交、引用关系不完整）时，再次插入会撞上唯一约束。两次运行之间的差异正好落在这个窗口里，因此崩溃并非每次都复现——这也解释了为什么「重试一次就好了」，但问题并未真正消失。

四、根因机理

4.1 代码缺陷

修复前的 run_colmap_sparse（节选）：

```
os.makedirs(workspace, exist_ok=True)
db = os.path.join(workspace, "database.db")
sparse = os.path.join(workspace, "sparse")
os.makedirs(sparse, exist_ok=True)
# ← 没有任何清理动作，直接复用可能存在的 database.db
matcher_cmd = _matcher_cmd(matcher)
steps = [[colmap, "feature_extractor", "--database_path", db, ...], ...]
```

只要 **99_runs/sfm_tmp/<item>/database.db** 存在，下一次全量重建就会在它上面继续追加。由此派生三个后果：

后果	表现	影响
约束冲突	重复插入 camera/image 撞唯一约束	COLMAP abort，0xC0000409
单相机失效	新会话各自创建相机（camera 1/2/3）	mapper 按相机分组，多子重建
不可复现	同一批照片两次运行结果不同	论文实验数据不可用

4.2 为什么 clean rebuild 也可能「看起来」成功

mapper 会把共享 3D 点的图像归入同一子重建。本次残留库虽然把图像分成camera 1 与 camera 3，但两组相机内参几乎相同（f=1805.08 vs 1804.04），共享点足够，最终仍能 30/30 注册。也就是说：**注册率看起来正常，但相机模型已被污染。**这类「隐性错误」比直接崩溃更危险——它会悄悄进入后续纹理投影与精度评估。

五、修复方案

5.1 全量重建前清库（核心）

新增 _reset_sparse_state()，在 run_colmap_sparse 开头调用；删除 database.db 及 -wal / -shm / -journal，清空 sparse/ 下旧模型，**但保留 photos_small**（照片预处理成果，避免每次重跑重新缩放）。

```
def _reset_sparse_state(db: str, sparse: str) -> None:
    for suffix in ("", "-wal", "-shm", "-journal"):
        p = db + suffix
        if os.path.exists(p):
            os.remove(p)        # 半写入的 WAL 一并清除
    if os.path.isdir(sparse):
        for name in os.listdir(sparse):
            p = os.path.join(sparse, name)
            if os.path.isdir(p):
                shutil.rmtree(p, ignore_errors=True)
            else:
                os.remove(p)
```

> 边界：增量注册是**另一条流程**（run_incremental_sparse），它需要保留旧库以追加新视角，因此本清理只在**全量重建**路径执行。photos_small 不清理——否则每次重跑都要重新解码 30 张单反原图。

5.2 COLMAP 输出完整落盘

新增 _dump_step_log()：每步写 logs/colmap_stepN.log，内容含完整命令行、退出码、stdout 与 stderr。修复前只把 stderr 末尾 600 字符塞进 failures.csv，用户无法事后排查。

5.3 失败信息可诊断

_error_tail() 提取尾部 14 行（去掉空行），并按失败特征追加建议：constraint failed → 数据库残留；0xC0000409 → 半写入/并发；CUDA → 驱动或构建；路径缺失 → 配置检查。同时给出日志绝对路径。

5.4 失败阶段记录真实耗时

RunManifest.record_fail() 增加 elapsed_s 参数，_isolate() 记录从阶段开始到失败的实际秒数。修复前恒为 0.00，既掩盖了「卡很久才失败」，也让耗时统计失真。

5.5 修复清单

文件	改动	作用
src/h3d/sfm_solve.py	_reset_sparse_state / _dump_step_log / _error_tail; run_colmap_sparse 与 dense 接入	清库 + 可诊断
src/h3d/core/metadata.py	record_fail(elapsed_s=...)	失败耗时
src/h3d/runner/batch_processor.py	_isolate 记录耗时	失败耗时
src/h3d/report/html_export.py	S1 入表、章节重编号、pre-wrap、诊断建议、统计横幅	报表可读性
src/h3d/report/csv_export.py	failures.csv 增加「耗时(s)」	报表可读性
src/h3d/gui/app.py	预览回退中间产物；失败提示；按实际文件启用按钮	GUI 可视化

文件	改动	作用
src/h3d/gui/preview.py	show_text()；matplotlib 中文字体	GUI 可视化

六、验证证据

6.1 干净库 S1 全流程实测

用修复后的代码、在全新工作区跑完整 S1（feature → match → mapper）：

```
11:12:16  S1 COLMAP[1/3] feature_extractor 开始
11:14:04  S1 COLMAP[1/3] 完成
11:14:04  S1 COLMAP[2/3] exhaustive_matcher 开始
11:19:07  S1 COLMAP[2/3] 完成
11:19:07  S1 COLMAP[3/3] mapper 开始
11:19:55  S1 COLMAP[3/3] 完成
EXIT=0   总耗时 7 分 41 秒
```

指标	污染库（修复前）	干净库（修复后）	判定
注册图像	30 / 30	30 / 30	持平
相机数	3（实际引用 2 个）	1	修复
3D 点数	13 971	13 990	略优
运行稳定	首次崩溃 0xC0000409	一次通过	修复
可复现	否（依赖历史状态）	是（每次从零）	修复

6.2 日志落盘实测

```
logs/colmap_step1.log   17040 B   feature_extractor
logs/colmap_step2.log     941 B   exhaustive_matcher
logs/colmap_step3.log   13989 B   mapper
```

6.3 回归测试

新增 tests/test_runner/test_sfm_reset.py（11 项）与 tests/test_report/test_report_failure.py（8 项），覆盖：清库范围、photos_small 不被误删、失败信息含日志路径与诊断、S1 进入状态表、章节编号唯一、失败信息保留换行、耗时真实记录、CSV 新增列。全量测试结果见交付说明。

6.4 失败报表实测渲染

用 S1 失败场景构造 manifest 并渲染 HTML，确认：状态表出现 S1 行与原因摘要、耗时显示 96.40、章节为一~六且无重复、诊断建议给出两条可执行建议。

七、报表与 GUI 可读性改造

7.1 报表前后对比

位置	修复前	修复后
一、状态表	只遍历 M1~M6，S1 失败时完全空白	纳入 S1/P1；新增「产物 / 异常摘要」列
顶部摘要	只有项目名与配置哈希	增加成功/失败/未执行统计与整体判定横幅
失败清单	无耗时列；换行被吞成一行	含耗时；pre-wrap 保留换行
诊断建议	无	按失败特征生成，最多 4 条
章节编号	两个「五、」	一~六连续唯一

7.2 GUI 前后对比

场景	修复前	修复后
S1 失败后预览区	空白，无任何提示	显示失败阶段与原因首行
存在中间产物时	不加载（只看 80_export）	回退加载 30_dense / 10_dense / fused.ply
失败后日志区	只有一句 summary	原因 + report.html 路径 + COLMAP 日志目录
「打开 open3d」按钮	失败也启用，点击报错	仅在存在可打开文件时启用

八、本次实机结果的判读与后续操作建议

用户第二次运行（断点续跑）在残留库上完成 S1，注册 30/30，目前正在执行 dense。需要明确两点：

问题	判读
为什么第二次能跑通?	残留库已经写完（30 张齐全），COLMAP 直接跳过已有图像，不再触发插入冲突。属于「碰巧避开」，不是问题消失。
这次结果能用吗?	几何可用，但相机模型有 2 个且参数略有差异，对纹理投影与精度评估不利。建议用修复版重跑一次干净链路。

8.1 建议操作步骤

① 用 r4 版 H3D.exe 打开同一项目；
② 不需要手动删除任何文件——系统会在 S1 开始时自动清理旧库与旧模型；
③ 若之前勾选了「断点续跑」，请取消，确保 S1 真正重跑；
④ 运行结束后查看 report.html：状态表应逐行显示 S1→M1→…→M6 的状态与耗时；
⑤ 若仍失败，直接查看 99_runs/sfm_tmp/<文物编号>/logs/colmap_stepN.log。

8.2 采集侧建议（30 张偏少）

本批 30 张、场景环绕，S1 已能 30/30 注册，3D 点约 1.4 万。对兵马俑这类弱纹理、强反光的陶俑，建议提高到 **60~100 张**、相邻重叠 70% 以上，并避免闪光灯直射造成的高光漂移；这样 dense 融合点云与纹理覆盖率会明显改善。

九、工程约束沉淀

本次故障与修复可沉淀为四条硬性约束，已写入项目长期记忆：

编号	约束	原因
E-1	全量重建不得建立在残留状态之上：先清库再建库	半写入库会触发约束冲突，且污染相机模型
E-2	外部程序每步输出必须完整落盘，失败信息含日志绝对路径	截断的 600 字符无法事后定位
E-3	状态表必须覆盖真实执行顺序的全部阶段（含 S1/P1）	否则失败时表格空白，用户无法判断进度
E-4	「能跑通」不等于「正确」：注册率正常也可能相机模型已污染	必须核对 camera 数与分组

> 一句话总结：本次崩溃不是「COLMAP 处理不了这批照片」，而是「系统让 COLMAP 在上一次的残局上继续下棋」。修复的关键不在于加参数，而在于把**状态边界**划清楚。